

Ejercicio Demostrativo

Luis Antonio Castillo Javier

Tiempo estimado: 2 horas

Universidad San Carlos de Guatemala
Facultad de Ingeniería
Escuela de Ingeniería en Ciencias y Sistemas.
Segundo Semestre 2026
[Laboratorio - Introducción a la Programación y Computación 2 - Sección D]

1. Marco Formativo

1.1 Valores

Nombre del valor	¿Cómo se aplica en tu laboratorio?
Responsabilidad	Durante el ejercicio demostrativo de JSON en C# y ASP.NET Core, la responsabilidad se evidencia en la forma en que el estudiante maneja correctamente la serialización y deserialización de datos. Un error en el formato JSON puede provocar fallos en toda la aplicación, por lo que documentar cada paso, verificar los tipos de datos y probar cada endpoint con Postman refleja el compromiso profesional esperado.

1.2 Competencias

Tipo de Competencia	Redacción
Competencia General	Desarrollar aplicaciones web que utilicen JSON como formato de intercambio de datos entre el frontend (Razor Pages) y el backend (ASP.NET Core Web API), aplicando serialización, deserialización y comunicación HTTP segura.
Competencia Específica	Implementar serialización y deserialización de objetos C# a JSON usando System.Text.Json y Newtonsoft.Json, consumir una Web API REST desde Razor Pages mediante HttpClient, e interpretar correctamente los códigos de respuesta HTTP en el contexto de una aplicación cliente-servidor.

2. Palabras Clave

JSON (JavaScript Object Notation): Formato de texto ligero para el intercambio de datos, basado en pares clave-valor.

Serialización: Proceso de convertir un objeto C# en una cadena de texto JSON.

Deserialización: Proceso inverso a la serialización; convierte una cadena JSON en un objeto C#.

System.Text.Json: Librería nativa de .NET para serializar y deserializar JSON con alto rendimiento.

Newtonsoft.Json: Paquete NuGet de terceros, muy flexible, ampliamente usado en proyectos .NET legacy.

HttpClient: Clase de .NET para enviar solicitudes HTTP y recibir respuestas desde una Web API.

Razor Pages: Framework de ASP.NET Core para crear páginas web dinámicas del lado del servidor (frontend).

Códigos HTTP: Números de tres dígitos que el servidor devuelve para indicar el resultado de una solicitud (200 OK, 404 Not Found, etc.).

HTTPS/TLS: Versión segura de HTTP que cifra la comunicación entre cliente y servidor mediante certificados.

3. Resumen

Este ejercicio demostrativo presenta de forma guiada cómo trabajar con JSON en el ecosistema C# y ASP.NET Core, abarcando desde la sintaxis básica de JSON hasta la integración completa entre un frontend en Razor Pages y un backend en ASP.NET Core Web API. El propósito es que el estudiante comprenda cómo los datos viajan entre el cliente y el servidor en formato JSON, y cómo se manejan correctamente en código C#.

La clase se orienta a que el estudiante comprenda la diferencia práctica entre JSON y XML en .NET, aplique serialización y deserialización con ambas librerías principales, use HttpClient para consumir una API REST desde Razor Pages, e interprete y maneje los códigos de respuesta HTTP que puede devolver un servidor ASP.NET Core.

4. Contenido

Descripción del ejercicio

El ejercicio consiste en construir una solución completa en ASP.NET Core compuesta por dos proyectos: una Web API REST que expone una lista de productos en formato JSON, y una aplicación de Razor Pages que consume esa API usando HttpClient y muestra los datos en una página web. A lo largo del ejercicio se practican la serialización/deserialización de objetos C# con System.Text.Json, el uso correcto de métodos HTTP y la interpretación de códigos de respuesta.

Objetivo de la demostración

Demostrar el procedimiento para serializar y deserializar objetos C# en JSON y consumir una Web API REST desde Razor Pages, utilizando System.Text.Json y HttpClient, con el fin de que el estudiante comprenda cómo se integran el frontend y el backend en una arquitectura ASP.NET Core moderna.

Recursos necesarios

- Software: .NET SDK 8.0 o superior, Visual Studio 2022 o Visual Studio Code.
- Herramientas técnicas: Postman o Thunder Client para probar los endpoints de la Web API; navegador web para verificar Razor Pages.
- Paquetes NuGet: Newtonsoft.Json (para demostrar la alternativa a System.Text.Json). Instalar con: `dotnet add package Newtonsoft.Json`.
- Archivos de apoyo: Presentación de la Semana 11, documentación de Microsoft Learn sobre System.Text.Json y HttpClient.
- Equipo necesario: Computadora con al menos 8 GB de RAM y conexión a internet para descargar paquetes NuGet.

Desarrollo paso a paso

Nota: El procedimiento está organizado para ser reproducible por cualquier tutor futuro. Seguir el orden indicado evita errores de referencias entre proyectos.

Paso 1 — Crear el modelo y demostrar serialización básica

Crear un proyecto de consola (`dotnet new console -n JsonDemo`) para demostrar serialización antes de integrar la API. Dentro del proyecto, crear la clase `Producto` con propiedades `Id (int)`, `Nombre (string)` y `Precio (decimal)`. Mostrar al estudiante cómo serializar con `System.Text.Json`: `string json = JsonSerializer.Serialize(producto);` y cómo deserializar: `var obj = JsonSerializer.Deserialize<Producto>(json);`. Repetir el mismo ejemplo con `Newtonsoft.Json`: `JsonConvert.SerializeObject(producto)` y `JsonConvert.DeserializeObject<Producto>(json)`. El

estudiante debe observar que la salida JSON es idéntica en ambos casos, pero la API de uso es distinta.

Paso 2 — Crear la Web API REST con endpoint JSON

Crear un nuevo proyecto Web API (`dotnet new webapi -n ProductosAPI`). Dentro de la carpeta `Models`, crear la clase `Producto`. En la carpeta `Controllers`, crear `ProductosController.cs` con los atributos `[ApiController]` y `[Route("api/[controller]")]`. Implementar el método `[HttpGet]` que retorne `Ok(lista)` con una lista de productos en memoria. Ejecutar con `dotnet run` y verificar desde Postman que `GET /api/productos` devuelve un arreglo JSON con código 200 OK. Mostrar al estudiante la cabecera `Content-Type: application/json` que ASP.NET Core agrega automáticamente.

Paso 3 — Crear Razor Pages y consumir la API con HttpClient

Crear un proyecto de Razor Pages (`dotnet new razor -n ProductosFrontend`). Registrar `HttpClient` en `Program.cs`: `builder.Services.AddHttpClient()`. En el `PageModel` de la página `Index`, inyectar `HttpClient` en el constructor y en el método `OnGetAsync()` hacer: `Items = await _http.GetFromJsonAsync<List<Producto>>("https://localhost:XXXX/api/productos");`. En el archivo `.cshtml` mostrar los productos con un `@foreach`. Verificar que la página carga correctamente y muestra los datos recibidos desde la API. El estudiante debe observar cómo `GetFromJsonAsync<T>()` deserializa automáticamente el JSON de la respuesta.

Paso 4 — Interpretar códigos HTTP y manejar errores

Extender el controlador de la API para incluir `GetById(int id)`: si el producto existe, retornar `Ok(producto)` (200); si no existe, retornar `NotFound()` (404). En la Razor Page, envolver la llamada con `try-catch` y verificar la propiedad `IsStatusCode` del `HttpResponseMessage`. Demostrar qué ocurre cuando se hace una solicitud a un id inexistente y cómo el frontend puede manejar el 404 sin romper la aplicación. Confirmar que el ejercicio está completo cuando la página muestra los productos correctamente y maneja el error 404 con un mensaje amigable.

Observaciones importantes

- Error muy común: no registrar `HttpClient` en `Program.cs` antes de inyectarlo en el `PageModel`. Esto genera una excepción en tiempo de ejecución, no en compilación.
- Al usar `GetFromJsonAsync<T>()`, si el servidor devuelve un código de error (4XX o 5XX), el método lanza una `HttpRequestException`. Siempre usar `try-catch` o verificar `IsStatusCode` primero.
- En .NET 8, los nombres de propiedades JSON son `camelCase` por defecto con `System.Text.Json`. Si la API devuelve 'nombre' y el modelo tiene 'Nombre', asegurarse de configurar `JsonSerializerOptions` o usar `[JsonPropertyName]`.
- No mezclar `System.Text.Json` y `Newtonsoft.Json` en el mismo proyecto sin una razón justificada, ya que puede generar conflictos de configuración.
- Para la demo de Razor Pages consumiendo la API, ambos proyectos deben estar corriendo simultáneamente en puertos distintos.

Tabla 1 — Comparativa JSON vs XML en el ecosistema .NET

Criterio	JSON (System.Text.Json)	XML (System.Xml)
Tamaño del payload	Compacto, menor latencia	Más voluminoso por las etiquetas
Legibilidad	Alta, sintaxis simple	Media, muchas etiquetas
Serialización en .NET	JsonSerializer.Serialize<T>()	XmlSerializer.Serialize()
Uso recomendado	REST APIs, Razor Pages, móviles	Sistemas legacy, SOAP, documentos complejos
Soporte ASP.NET Core	Nativo, sin configuración extra	Requiere configuración adicional

Resultado esperado

Al finalizar el ejercicio, el estudiante debe tener en funcionamiento dos proyectos ASP.NET Core: una Web API REST que serializa una lista de productos a JSON y responde con códigos HTTP correctos (200 OK, 404 Not Found), y una aplicación Razor Pages que consume esa API mediante HttpClient, deserializa la respuesta JSON y muestra los productos en una página HTML. Ambos proyectos deben compilar y ejecutarse sin errores, y la comunicación entre ellos debe ser verificable desde el navegador.

Aplicación práctica

Este ejercicio representa el patrón de arquitectura más usado en el desarrollo web moderno con .NET: un backend como Web API que expone datos en JSON y un frontend (Razor Pages, React, Angular o móvil) que los consume mediante HTTP. Los conceptos de serialización, HttpClient y códigos de respuesta HTTP son fundamentales en cualquier proyecto profesional de desarrollo de software, y la comprensión de JSON como formato universal de intercambio de datos es indispensable para integraciones con servicios externos, APIs de terceros y microservicios.

5. Aconsejando al siguiente Tutor para la realización de ejemplos

- Dificultades encontradas: La mayor dificultad fue la configuración simultánea de dos proyectos (API + Razor Pages) corriendo en la misma máquina. Varios estudiantes confundieron los puertos y apuntaban el HttpClient al puerto incorrecto. Se recomienda preparar y publicar los URLs de ambos proyectos en la pizarra antes de comenzar.
- Recomendaciones para futuras sesiones: Preparar el ejemplo de serialización con consola por separado antes de entrar a la integración Razor Pages + API. Así el estudiante entiende JSON de forma aislada antes de ver la comunicación completa. También es útil mostrar en tiempo real la solicitud y respuesta en Postman con la pestaña de respuesta visible.
- Puntos de mayor dificultad para el estudiante: La diferencia entre `GetFromJsonAsync<T>()` y `ReadFromJsonAsync<T>()` causó confusión. Se recomienda mostrar ambos métodos y explicar cuándo usar cada uno. También el concepto de que HttpClient debe registrarse como servicio y no instanciarse manualmente (para evitar agotamiento de sockets) fue difícil de asimilar; dedicar un par de minutos a explicar el patrón `IHttpClientFactory` resulta muy útil.

6. Referencias

1. Microsoft. (2024). System.Text.Json overview. Microsoft Learn. <https://learn.microsoft.com/en-us/dotnet/standard/serialization/system-text-json/overview>
2. Microsoft. (2024). ASP.NET Core Web API. Microsoft Learn. <https://learn.microsoft.com/en-us/aspnet/core/web-api>
3. Microsoft. (2024). Razor Pages in ASP.NET Core. Microsoft Learn. <https://learn.microsoft.com/en-us/aspnet/core/razor-pages>
4. Newtonsoft. (2024). Json.NET Documentation. <https://www.newtonsoft.com/json/help/html/Introduction.htm>
5. Códigos de estado HTTP - HTTP | MDN. (2025). MDN Web Docs. <https://developer.mozilla.org/es/docs/Web/HTTP/Reference/Status>
6. GeeksforGeeks. (2025). HTTP Full Form. <https://www.geeksforgeeks.org/http-full-form/>